

Lab 03 – Worksheet

Name: M. Shees Abbas	ID: ma11212	Section:T4
----------------------	-------------	------------

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

Provide appropriately commented codes (Mention question part before each part)

```
.text
.globl main
main:
    addi x10, x0, 12
    addi x11, x0, 12
    jal x1, sum
    addi x11,x10, 0
    li x10,1
    ecall
    j Exit
sum:
    add x10, x11,x10
    jalr x0,0(x1)

Exit:

end:
    j end
```

Add screenshot of your results (register)

PC before and after procedure call:

PC = 0x00000008 PC = 0x0000001C

PC before and after procedure return:

PC = 0x00000020 PC = 0x0000000C

Terminal output from ecall:

```
-----
-----
Starting program C:\Users\ma11212\CA lab\CA-labs\lab_3\task1.s
24
```

The final values of registers:

x00 (zero) = 0x00000000	x17 (a7) = 0x00000000
x01 (ra) = 0x0000000C	x18 (s2) = 0x00000000
x02 (sp) = 0x7FFFFFF0	x19 (s3) = 0x00000000
x03 (gp) = 0x10000000	x20 (s4) = 0x00000000
x04 (tp) = 0x00000000	x21 (s5) = 0x00000000
x05 (t0) = 0x00000000	x22 (s6) = 0x00000000
x06 (t1) = 0x00000000	x23 (s7) = 0x00000000
x07 (t2) = 0x00000000	x24 (s8) = 0x00000000
x08 (s0) = 0x00000000	x25 (s9) = 0x00000000
x09 (s1) = 0x00000000	x26 (s10) = 0x00000000
x10 (a0) = 0x00000001	x27 (s11) = 0x00000000
x11 (a1) = 0x00000018	x28 (t3) = 0x00000000
x12 (a2) = 0x00000000	x29 (t4) = 0x00000000
x13 (a3) = 0x00000000	x30 (t5) = 0x00000000
x14 (a4) = 0x00000000	x31 (t6) = 0x00000000
x15 (a5) = 0x00000000	
x16 (a6) = 0x00000000	

Task 2

Provide appropriately commented codes (Mention question part before each part)

```
.text
.globl main
main:
    li x10, 4
```

```
li x11, 3
li x12, 2
li x13, 1

li x18, 10
li x19, 20
li x20, 30

jal x1, leaf
j Exit

leaf:
    addi sp, sp, -12
    sw x18, 0(sp)
    sw x19, 4(sp)
    sw x20, 8(sp)
    li x20, 0
    add x18, x10, x11
    add x19, x12, x13
    sub x20, x18, x19
    mv x10, x20
    lw x20, 8(sp)
    lw x19, 4(sp)
    lw x18, 0(sp)
    addi sp, sp, 12
    jalr x0, 0(x1)

Exit:

end:
    j end
```

Add screenshot of your results

PC before and after procedure call:

PC = 0x0000001C PC = 0x00000024

Sp after storing registers in stack and after loading from stack:

x02 (sp) = 0x7FFFFFFE4 x02 (sp) = 0x7FFFFFFF0

Memory values:

0x7FFFFFF4	00	00	00	00
0x7FFFFFF0	00	00	00	00
0x7FFFFFEC	1E	00	00	00
0x7FFFFE8	14	00	00	00
0x7FFFFE4	0A	00	00	00
0x75555550	00	00	00	00

Final register

x00 (zero) = 0x00000000	
x01 (ra) = 0x00000020	
x02 (sp) = 0x7FFFFFF0	x17 (a7) = 0x00000000
x03 (gp) = 0x10000000	x18 (s2) = 0x0000000A
x04 (tp) = 0x00000000	x19 (s3) = 0x00000014
x05 (t0) = 0x00000000	x20 (s4) = 0x0000001E
x06 (t1) = 0x00000000	x21 (s5) = 0x00000000
x07 (t2) = 0x00000000	x22 (s6) = 0x00000000
x08 (s0) = 0x00000000	x23 (s7) = 0x00000000
x09 (s1) = 0x00000000	x24 (s8) = 0x00000000
x10 (a0) = 0x00000004	x25 (s9) = 0x00000000
x11 (a1) = 0x00000003	x26 (s10) = 0x00000000
x12 (a2) = 0x00000002	x27 (s11) = 0x00000000
x13 (a3) = 0x00000001	x28 (t3) = 0x00000000
x14 (a4) = 0x00000000	x29 (t4) = 0x00000000
x15 (a5) = 0x00000000	x30 (t5) = 0x00000000
x16 (a6) = 0x00000000	x31 (t6) = 0x00000000

Task 3 Provide appropriately commented codes (Mention question part before each part)

```
.text
.globl main

main:
    li    a0, 0x100          # a0 = base address of v
    li    a1, 0              # a1 = k = 0

    # Populate only the two elements the swap uses
    li    t0, 10
    li    t1, 20
    sw    t0, 0(a0)          # v[0] = 10
    sw    t1, 4(a0)          # v[1] = 20

    jal   ra, swap           # call swap(v, k)

    lw    s0, 0(a0)          # s0 = v[0], should now be 20
    lw    s1, 4(a0)          # s1 = v[1], should now be 10
    j     end

swap:
    slli  t1, a1, 2           # t1 = k * 4 (each long is 4 bytes)
```

```

add t1, a0, t1      # t1 = address of v[k]
lw  t0, 0(t1)       # temp = v[k]
lw  t2, 4(t1)       # t2 = v[k+1]
sw  t2, 0(t1)       # v[k] = v[k+1]
sw  t0, 4(t1)       # v[k+1] = temp
jalr x0, 0(ra)      # return to caller

end:
j    end

```

Add screenshots of your results

PC before and after function call

PC = 0x00000018 PC = 0x00000028

Memory before and after function call

0x00000104	14	00	00	00
0x00000100	0A	00	00	00

0x00000104	0A	00	00	00
0x00000100	14	00	00	00

Final register values

x00 (zero) = 0x00000000	x18 (s2) = 0x00000000
x01 (ra) = 0x0000001C	x19 (s3) = 0x00000000
x02 (sp) = 0x7FFFFFF0	x20 (s4) = 0x00000000
x03 (gp) = 0x10000000	x21 (s5) = 0x00000000
x04 (tp) = 0x00000000	x22 (s6) = 0x00000000
x05 (t0) = 0x0000000A	x23 (s7) = 0x00000000
x06 (t1) = 0x00000100	x24 (s8) = 0x00000000
x07 (t2) = 0x00000014	x25 (s9) = 0x00000000
x08 (s0) = 0x00000014	x26 (s10) = 0x00000000
x09 (s1) = 0x0000000A	x27 (s11) = 0x00000000
x10 (a0) = 0x00000100	x28 (t3) = 0x00000000
x11 (a1) = 0x00000000	x29 (t4) = 0x00000000
x12 (a2) = 0x00000000	x30 (t5) = 0x00000000
x13 (a3) = 0x00000000	x31 (t6) = 0x00000000
x14 (a4) = 0x00000000	
x15 (a5) = 0x00000000	
x16 (a6) = 0x00000000	
x17 (a7) = 0x00000000	

Task 4 Provide appropriately commented codes

```
.text
.globl main

main:
    li    a0, 0x200          # a0 = base address of x (destination)
    li    a1, 0x100          # a1 = base address of y (source)

    # Populate y with the string "Hi!" and the null terminator
    li    t0, 72              # 'H'
    li    t1, 105             # 'i'
    li    t2, 33              # '!'
    sb     t0, 0(a1)          # y[0] = 'H'
    sb     t1, 1(a1)          # y[1] = 'i'
    sb     t2, 2(a1)          # y[2] = '!'
    sb     zero, 3(a1)        # y[3] = '\0'

    jal    ra, strcpy         # call strcpy(x, y)

    lbu    s0, 0(a0)          # s0 = x[0], should be 72 ('H')
    lbu    s1, 1(a0)          # s1 = x[1], should be 105 ('i')
    lbu    s2, 2(a0)          # s2 = x[2], should be 33 ('!')
    lbu    s3, 3(a0)          # s3 = x[3], should be 0 ('\0')
    j      end                # done

strcpy:
    li     t0, 0              # i = 0
loop:
    add     t1, t0, a1         # t1 = address of y[i]
    lbu     t2, 0(t1)          # t2 = y[i]
    add     t3, t0, a0         # t3 = address of x[i]
    sb      t2, 0(t3)          # x[i] = y[i]
    beq     t2, zero, done     # if y[i] == '\0', exit loop
    addi    t0, t0, 1          # i += 1
    j      loop
done:
    jalr    x0, 0(ra)

end:
    j      end
```

Add screenshots of your results

Memory value before and after function call

0x00000108	00	00	00	00	0x00000208	00	00	00	00
0x00000104	00	00	00	00	0x00000204	00	00	00	00
0x00000100	48	69	21	00	0x00000200	00	00	00	00

After:

0x00000108	00	00	00	00	0x00000208	00	00	00	00
0x00000104	00	00	00	00	0x00000204	00	00	00	00
0x00000100	48	69	21	00	0x00000200	48	69	21	00

Final register values:

x00 (zero) = 0x00000000	x18 (s2) = 0x00000021
x01 (ra) = 0x00000028	x19 (s3) = 0x00000000
x02 (sp) = 0x7FFFFFF0	x20 (s4) = 0x00000000
x03 (gp) = 0x10000000	x21 (s5) = 0x00000000
x04 (tp) = 0x00000000	x22 (s6) = 0x00000000
x05 (t0) = 0x00000003	x23 (s7) = 0x00000000
x06 (t1) = 0x00000103	x24 (s8) = 0x00000000
x07 (t2) = 0x00000000	x25 (s9) = 0x00000000
x08 (s0) = 0x00000048	x26 (s10) = 0x00000000
x09 (s1) = 0x00000069	x27 (s11) = 0x00000000
x10 (a0) = 0x00000200	x28 (t3) = 0x00000203
x11 (a1) = 0x00000100	x29 (t4) = 0x00000000
x12 (a2) = 0x00000000	x30 (t5) = 0x00000000
x13 (a3) = 0x00000000	x31 (t6) = 0x00000000
x14 (a4) = 0x00000000	
x15 (a5) = 0x00000000	
x16 (a6) = 0x00000000	
x17 (a7) = 0x00000000	

Lab 03 – Introduction to RISC V Assembly (Jumps and Returns)

Assessment Rubrics

Points Distribution

Points Distribution			
Task No.	LR2 Code	LR5 Results	AR7 Report Submission and Git Upload
Task 1	/10	/05	/10 & /10
Task 2	/10	/10	
Task 3	/10	/10	
Task 4	/15	/10	
Total Points	/100 Points		
CLO Mapped	CLO 2		

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3: Good Points 3	Level 4: Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures / plots are not included. Report is submitted after due date.	Good summary of most the in-lab tasks is included in report. The work is supported by figures and plots with explanations. The report is submitted timely.	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly including all the necessary figures, plots and tables.